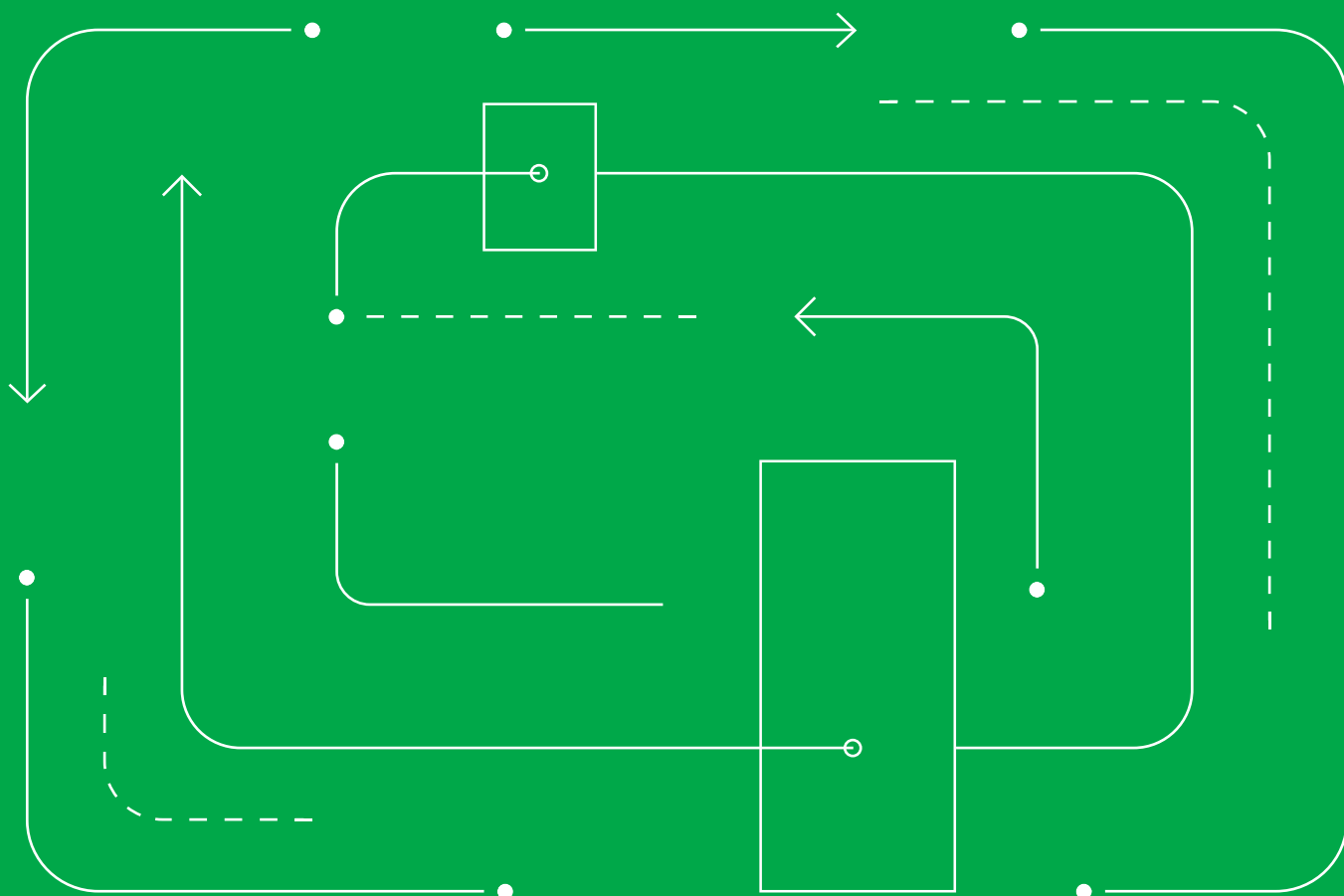


Введение в SQL



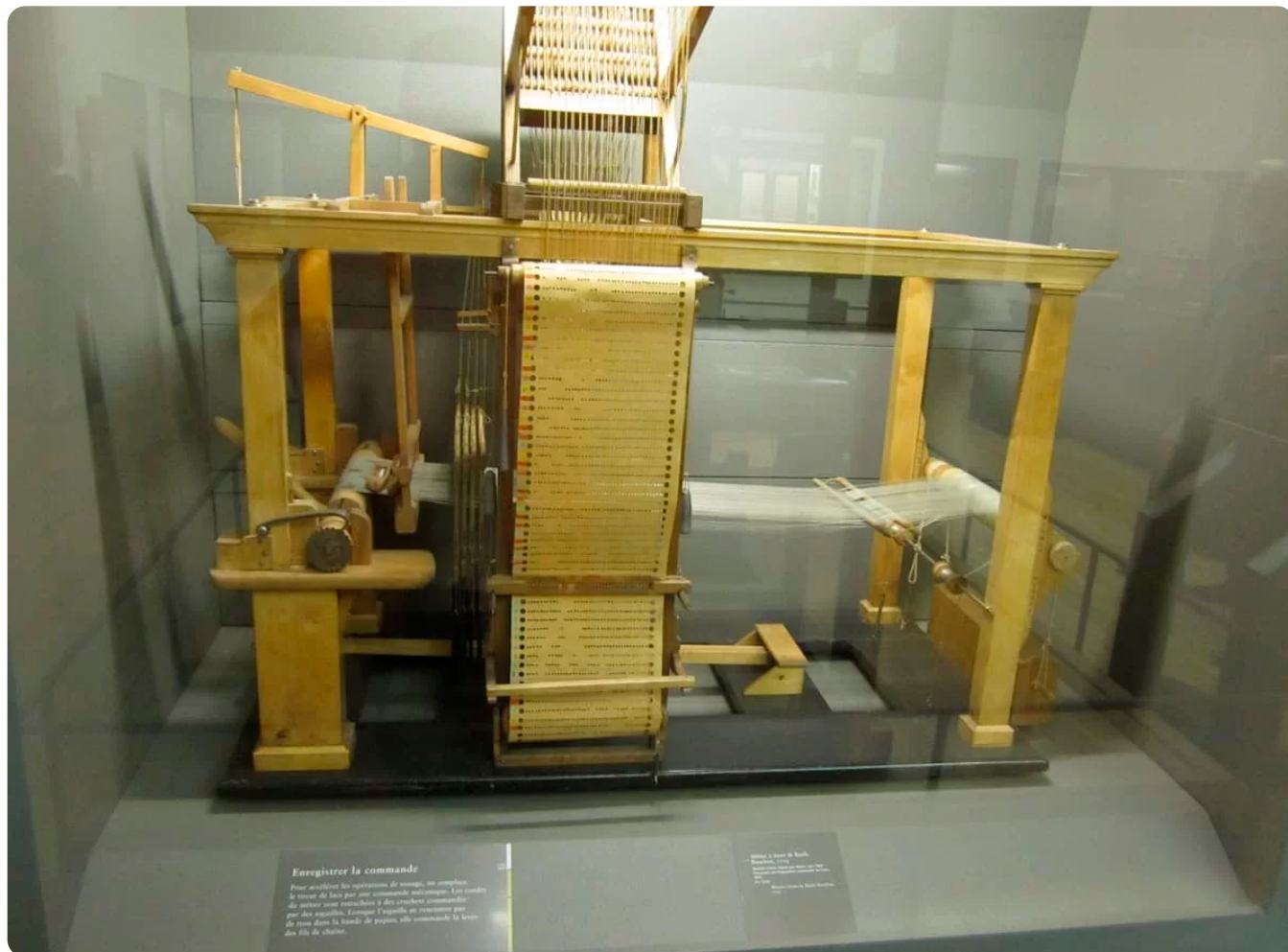
SQL И БАЗЫ ДАННЫХ (ДЛЯ РАЗРАБОТЧИКОВ)

ЛОНГРИД

НЕДЕЛЯ 1

Немного истории

Системы хранения данных (СХД) прошли впечатляющий путь развития. За последние 60–70 лет они эволюционировали от простейших механических устройств до современных высокотехнологичных накопителей. Сегодняшние СХД — это не только результат стремительного научно-технического прогресса, но и свидетельство того, как инновации меняют наш мир.



Ткацкий станок Базиля Бушона, XIX век

История СХД началась задолго до появления первых компьютеров. В начале XIX века, в эпоху текстильной промышленности, французский ткач Базиль Бушон изобрёл перфорированную ленту. Этот носитель служил для управления ткацким станком, записывая «программы» в виде отверстий. Простая, но гениальная идея позволила автоматизировать создание узоров на тканях. Работник прижимал ленту к полотну игл, и лишь те нити, которые совпадали с перфорацией, проходили сквозь станок, формируя узор. Это изобретение стало первым шагом на пути к современным методам хранения и обработки данных.

С тех пор прошло почти три столетия и технологии хранения данных изменились до неузнаваемости. XX век подарил миру множество разнообразных устройств: магнитные ленты, барабаны, жёсткие диски, гибкие диски, оптические диски и твердотельные накопители. Некоторые из них остались в прошлом, а другие, такие как накопители на основе твердотельной электроники, облачные хранилища и даже квантовые системы, продолжают развиваться, формируя будущее отрасли.

Современные СХД поражают своей ёмкостью, компактностью и скоростью. Объёмы данных, которые раньше занимали целые залы серверов, сегодня могут храниться на устройстве, помещающемся на ладони. Скорость передачи информации преодолела все мыслимые пределы, а расстояния больше не являются преградой. Человечество шагнуло в эпоху, где информация стала глобальным ресурсом, доступным каждому, и её роль в жизни общества трудно переоценить.

Подробнее прочесть об эволюции систем хранения данных можно [тут](#).

Базы данных

Базы данных везде вокруг нас. Любой сервис или приложение, которые ты используешь, хранит данные в своих базах данных: социальные сети хранят данные твоего профиля, историю сообщений, помнят, какие реакции и на какие сообщения тобой поставлены. И это нужно для корректной работы самого сервиса или приложения.

ТЕРМИН

Данные — это набор фактов, которые нужно хранить. Например, в магазине это может быть информация о товарах, клиентах и продажах.

Часто, когда мы говорим о базах данных, мы имеем в виду системы управления базами данных. Давай разбираться.

ТЕРМИН

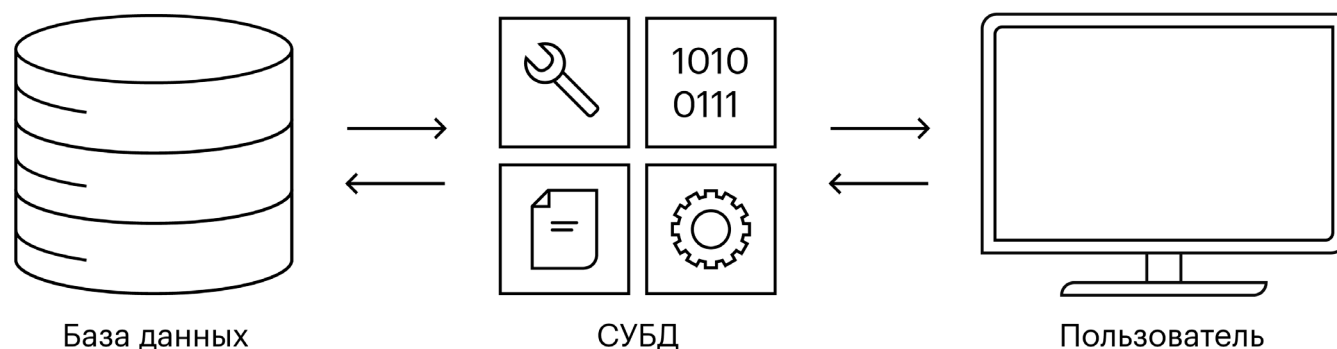
База данных (БД) — это организованный набор информации, хранящейся в электронном виде.

Фактически база данных представляет собой просто файл на диске компьютера, который умеет только хранить данные. И чтобы облегчить работу с БД, были придуманы системы управления базами данных (СУБД).

ТЕРМИН

Система управления базами данных (СУБД) — программное обеспечение, которое позволяет пользователю взаимодействовать с базой данных и даёт возможность, например, добавлять новые элементы, удалять старые, фильтровать и находить нужные элементы, менять их структуру, создавать резервные копии.

СУБД является связующим звеном между базой данных и пользовательскими запросами к ней.



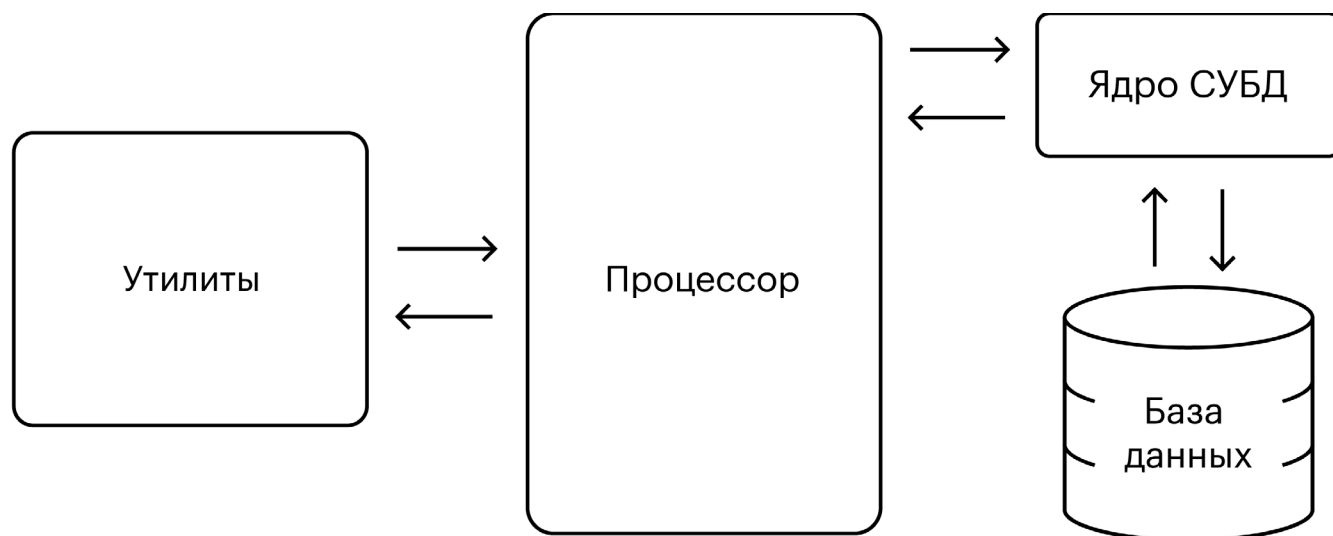
Если объяснять работу СУБД на примере онлайн-магазина, то получится следующая последовательность действий.

1. Пользователь открывает страницу со списком своих заказов и нажимает на кнопку «Посмотреть детали» для конкретного заказа.
2. Веб-приложение (сайт) отправляет запрос на сервер с идентификатором заказа (например: ID_Заказа = 12345).
3. Сервер преобразует запрос пользователя в SQL-запрос к базе данных.
4. На этом этапе подключается (образно выражаясь) СУБД. СУБД принимает запрос и выполняет его, используя базу данных:
 - ищет информацию о заказе в таблице «Заказы»;
 - находит клиента в таблице «Клиенты» по *ID_Клиента*;
 - находит товары, связанные с заказом, в таблице «Товары_В_Заказе» и их описание в таблице «Товары».
5. СУБД возвращает данные в виде таблицы.
6. Сервер преобразует результаты в JSON или другой формат, удобный для передачи веб-приложению.
7. На стороне клиента веб-приложение обрабатывает данные и показывает их пользователю в удобной форме.

ДЛЯ ЧЕГО НУЖНЫ СУБД?

1. **Управление данными.** СУБД позволяет создавать, изменять и удалять базы данных и их структуры.
2. **Доступ к данным.** Обеспечивает возможность добавления, обновления, удаления и извлечения данных из базы данных.
3. **Безопасность.** Управляет доступом пользователей к данным, обеспечивая их защиту и конфиденциальность.
4. **Резервное копирование и восстановление.** Позволяет создавать резервные копии данных и восстанавливать их в случае сбоев или потерь.
5. **Обеспечение целостности данных.** Поддерживает правила и ограничения, гарантирующие точность и согласованность данных.

ИЗ ЧЕГО СОСТОИТ СУБД?



Ядро. Отвечает за работу всей системы в целом. Через него проходят все процессы обработки данных и их хранения. Ядро следит за всеми изменениями баз данных и фиксирует их.

Процессор, или компилятор. Занимается обработкой запросов от пользователей. Главная его задача — преобразовать SQL-запрос в понятные для компьютера команды, а затем вернуть результаты.

Программные средства, или утилиты. Нужны, чтобы пользователь мог вводить запросы, а администраторы могли настраивать доступ и другие необходимые параметры.

Базы данных. Место, где хранятся данные в упорядоченном, а иногда и в зашифрованном виде. Базы могут различаться по структуре и типам представления данных.

ВИДЫ СУБД

Существует несколько классификаций СУБД на основе разных параметров. Мы же рассмотрим следующие.

1. По расположению баз данных.

- **Локальные.** Содержимое базы данных располагается на одном компьютере, обычно на сервере компании.
- **Распределённые.** База данных распределена по разным компьютерам, например по нескольким серверам.

2. По языку запросов.

- **SQL.** Язык структурированных запросов к базам данных. Такие СУБД самые популярные в использовании.
- **NoSQL.** Язык запросов, который основан на другом языке программирования, например Python или JavaScript. СУБД с NoSQL обычно используют при работе с большими данными.

3. По обработке данных.

Для того чтобы выделить одну из основных особенностей OLTP-систем, необходимо дать определение транзакциям в терминах баз данных.

ТЕРМИН

Транзакции в базах данных представляют собой набор операций, которые выполняются как единое целое, обеспечивая целостность и согласованность данных. Основная цель транзакций — гарантировать, что все изменения, внесённые в базу данных, либо будут применены полностью, либо не будут применены вовсе.

Параметр	OLTP	OLAP
Цель	Обработка транзакций в реальном времени	Анализ данных и поддержка принятия решений
Структура данных	Нормализованные таблицы	Денормализованные таблицы
Тип операций	Вставка, обновление, удаление	Чтение и агрегация
Примеры использования	Банковские системы, интернет-магазины	Бизнес-аналитика, отчётность
Производительность	Высокая при работе с транзакциями	Высокая при выполнении сложных запросов

OLTP

Это системы, предназначенные для обработки большого количества операций в реальном времени. Они используются для записи и управления транзакциями, такими как покупки, переводы денежных средств, обновление информации по заказу и так далее.

Основные характеристики

- **Простые запросы.** OLTP-системы прекрасно обрабатывают простые запросы в БД, такие как вставка, обновление и удаление данных. Более сложные запросы, например агрегацию или соединение таблиц, такие БД делают плохо.
- **Высокая скорость и доступность.** Эти системы обеспечивают быструю обработку запросов с минимальной задержкой, что критично для операций, требующих мгновенного ответа, например, при проведении финансовых транзакций.
- **Конкурентный доступ.** OLTP-системы обеспечивают управление конкурентным доступом к данным, предотвращая одновременные изменения одной и той же записи разными пользователями. Это достигается с помощью механизмов блокировок и контроля версий.
- **ACID-свойства.** Это набор свойств, который гарантирует, что все операции в рамках одной транзакции либо завершаются успешно, либо откатываются в случае ошибки.

Примеры применения

OLTP-системы широко используются в различных отраслях:

- **финансовые услуги** — онлайн-банкинг, банкоматы и системы обработки платежей;
- **электронная коммерция** — интернет-магазины и платформы для бронирования;
- **управление запасами** — системы учёта на складах и в магазинах.

OLAP

Это системы, специально разработанные для выполнения сложных аналитических запросов и многомерного анализа больших объёмов данных. Они обеспечивают высокую производительность при выполнении запросов на чтение.

Основные характеристики

- **Аналитическая обработка.** OLAP позволяет выполнять сложные запросы для анализа больших объёмов исторических данных. Это полезно для бизнес-аналитики и принятия стратегических решений.
- **Низкая частота обновлений.** В отличие от OLTP, данные в OLAP-системах обновляются реже. Основное внимание уделяется анализу и отчётности на основе уже собранных данных.
- **Оптимизация под чтение.** OLAP-системы оптимизированы для чтения данных и могут обрабатывать сложные запросы на анализ с использованием различных методов агрегации.

Примеры применения

OLAP-системы находят широкое применение в различных областях:

- **бизнес-аналитика** — поддержка принятия решений на основе анализа исторических данных;
- **финансовая отчётность** — позволяет компаниям быстро генерировать отчёты о доходах, расходах и других финансовых показателях;
- **прогнозирование продаж** — анализ тенденций для прогнозирования будущих продаж на основе исторических данных;
- **планирование бюджета** — помогает организациям в составлении бюджетов на основе анализа прошлых расходов и доходов.

4. По используемой модели данных

ТЕРМИН

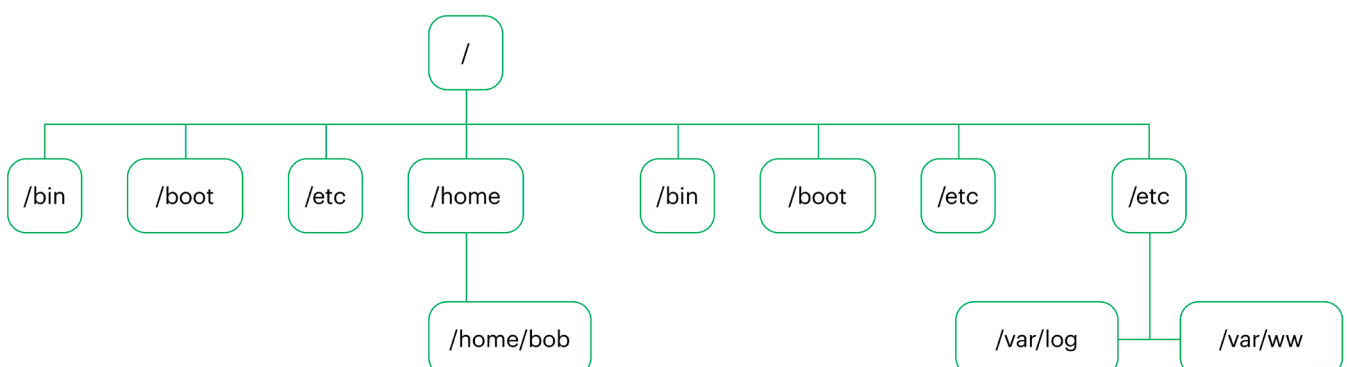
Модель данных — это способ организации и представления данных в базе данных, который определяет, как данные хранятся, структурируются и обрабатываются.

Выделяют несколько основных моделей данных.

1. Иерархическая модель

Это структура, организующая данные в виде древовидной схемы, где каждый элемент (узел) связан с одним родительским элементом и может иметь несколько дочерних элементов.

Эта модель была одной из первых в области управления базами данных и до сих пор используется в различных системах.



Преимущества

- **Простота концепции.** Иерархическая модель интуитивно понятна и легко воспринимается пользователями, что облегчает её реализацию и использование.
- **Эффективное использование памяти.** Модель позволяет эффективно использовать ресурсы системы, особенно при работе с большими объёмами данных.
- **Высокая скорость обработки.** Операции над данными выполняются быстро благодаря простой структуре и чётким связям между узлами.

Ограничения

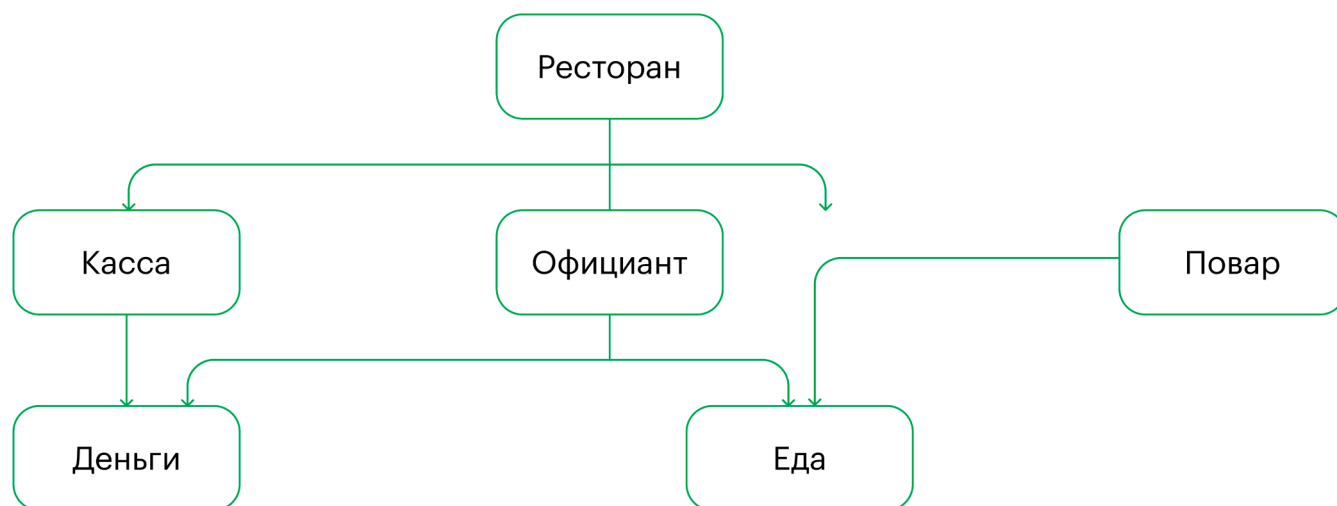
- **Ограниченная гибкость.** Из-за жёсткой структуры изменения в иерархии могут быть сложными. Например, перемещение узла в другую часть дерева требует значительных усилий.
- **Сложность обработки.** При необходимости выполнения сложных запросов или анализа данных возникают трудности из-за навигационного характера доступа к данным.

Примеры использования

- **Структура каталогов на жёстком диске.** Файлы организованы в папки, которые могут содержать другие папки.
- **Системный регистр Windows.** Хранит настройки системы в виде иерархии ключей и значений.
- **Организационные структуры.** Моделирование отношений между сотрудниками и отделами компании.

2. Сетевая модель

Это структура, которая организует данные в виде сетевых отношений, позволяя делать более сложные, по сравнению с иерархической моделью, связи между записями. Данные организованы в виде графа, где узлы представляют записи, а рёбра — связи между ними. Каждая запись может иметь несколько родительских и дочерних записей.



Преимущества

- **Гибкость в моделировании.** Возможность создания сложных структур данных и отношений между ними делает сетевую модель подходящей для многих приложений, где данные имеют взаимосвязи.
- **Эффективность.** Высокая скорость доступа к данным благодаря прямым указателям и возможности быстрого перемещения по структуре.

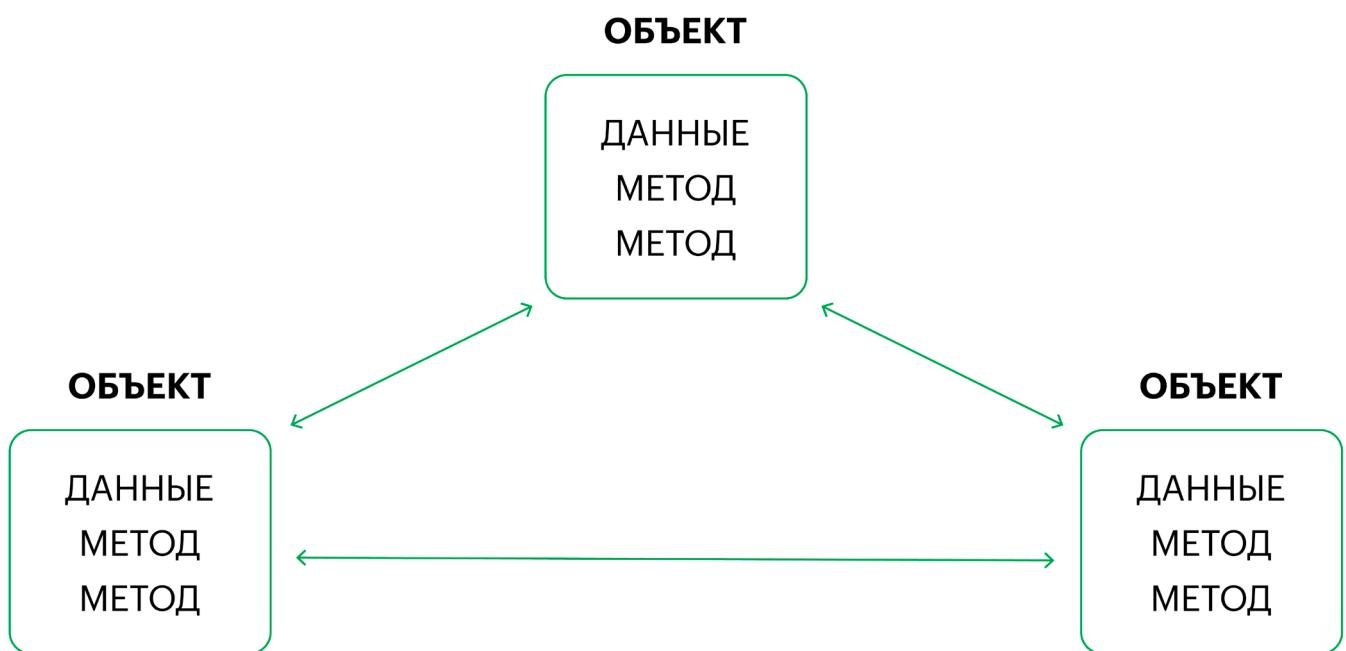
Ограничения

- **Сложность.** Сложная структура может затруднять понимание и управление данными, особенно для пользователей без технического образования.
- **Зависимость от структуры.** Изменение структуры данных может потребовать значительных изменений в приложениях, использующих эти данные, что делает систему менее гибкой по сравнению с реляционными моделями.

Эта модель была популярна в 1960–1980-х годах. Сейчас классическая сетевая модель устарела, её принципы нашли применение в графовых базах данных, которые мы рассмотрим ниже.

3. Объектно-ориентированная модель

Это подход к организации данных и управлению ими, который основывается на принципах объектно-ориентированного программирования (ООП). Объект имеет набор свойств, называемых атрибутами, и может включать в себя также процедуры для обработки данных, которые называют методами.



Преимущества

- **Гибкость.** Объектная модель позволяет легко моделировать сложные взаимосвязи между данными благодаря возможности вложения объектов друг в друга и установления многоуровневых отношений.
- **Повторное использование кода.** Объекты и классы можно многократно использовать в различных приложениях, что снижает затраты на разработку и упрощает поддержку программного обеспечения.
- **Поддержка мультимедийных данных.** Может работать с различными типами данных, включая текстовые, графические и аудио- или видеофайлы.

Ограничения

- **Нет стандартизированного языка запросов.** Используются специализированные API.
- **Непопулярность.** Объектные базы данных менее распространены, чем реляционные, из-за исторических причин и ограничений совместимости.
- **Плохая масштабируемость.** Масштабировать объектные базы сложнее из-за сложной структуры данных.

Примеры использования

- **1С: предприятие.** Объектами являются такие сущности, как «Клиент», «Заказ», «Товар». Каждый объект имеет свои атрибуты и методы.

4. Модель «Ключ — значение»

Одна из самых простых и широко используемых моделей в базах данных NoSQL. Она основывается на концепции хранения данных в виде пар «Ключ — значение», где каждый ключ уникален и служит для идентификации соответствующего значения. Эта модель особенно эффективна для приложений, требующих быстрого доступа к данным.

ключ	user_id:	f5badc33-5bd7-4b65-a737-b5304675f476
значение	color:	blue
	repetitions:	3
	text:	hello world
	data:	{...}

- Ключ

```
"user:1234"
```

- Значение

```
{"name": "Иван", "age": 30, "email": "ivan@example.com"}
```

Преимущества

- **Высокая производительность.** Быстрый доступ к данным делает эту модель идеальной для кеширования и быстрого поиска.
- **Гибкость.** Значения могут быть различных типов (строки, числа, JSON, бинарные данные), что позволяет хранить разнообразные данные.
- **Простота использования.** Простая структура упрощает разработку и обслуживание приложений.

Ограничения

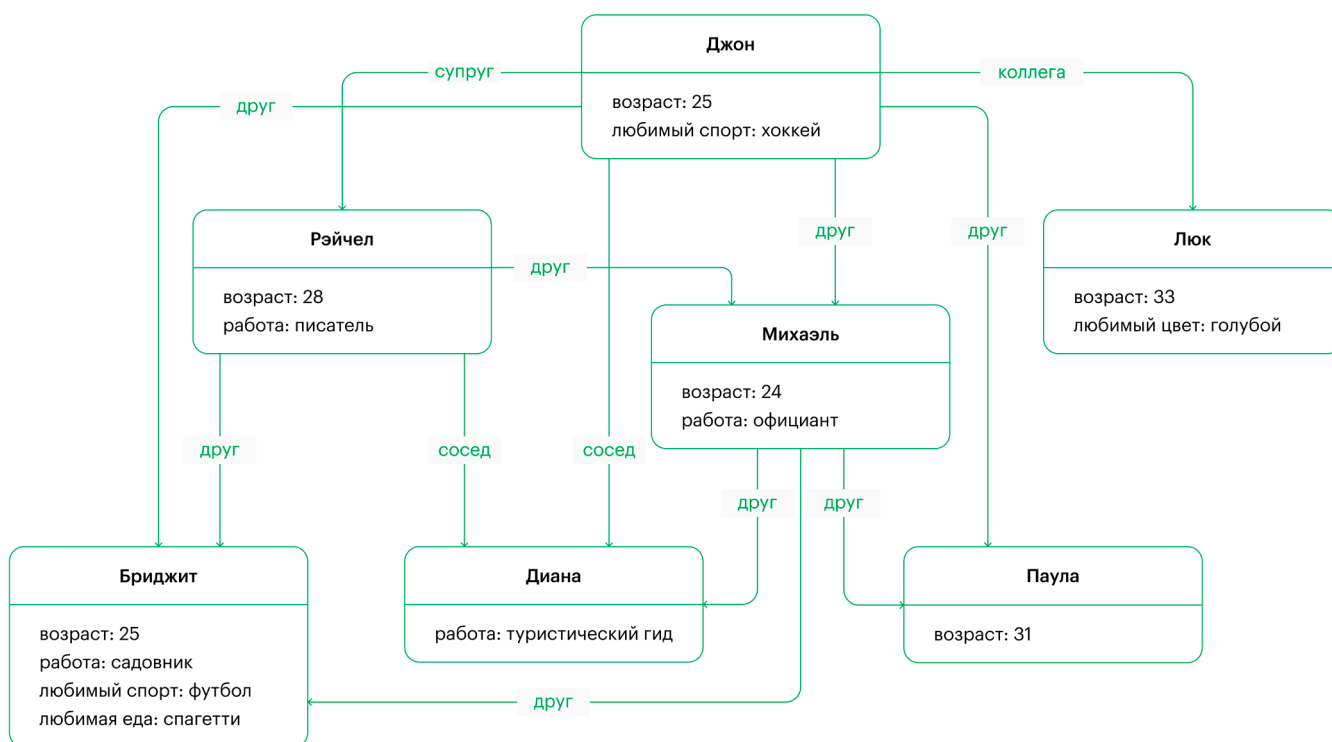
- **Отсутствие сложных запросов.** Базы данных «Ключ — значение» не поддерживают сложные запросы, такие как объединения или фильтрация, что может усложнить обработку данных.
- **Неэффективность при работе с группами записей.** При необходимости извлечения нескольких записей по критериям это может быть менее эффективно по сравнению с реляционными моделями.
- **Отсутствие языка запросов.** Логика обработки данных часто переносится в код приложения, что может усложнить поддержку.

Примеры использования

- **Кеширование.** Базы данных «Ключ — значение» часто используются для кеширования данных, таких как результаты запросов или временные данные.
- **Сессии пользователей.** Хранение информации о сессиях пользователей в веб-приложениях.
- **Электронная коммерция.** Хранение информации о товарах, корзинах покупок и заказах.

5. Графовая модель

Это способ организации и хранения информации в виде графа, где данные представлены в виде узлов (вершин) и рёбер (связей) между ними. Эта модель позволяет эффективно моделировать сложные взаимосвязи между данными, что делает её особенно полезной в сценариях, где важны отношения между объектами.



Пример графа, представляющего социальную сеть

(Вершины)

Пользователь (ID: 1, Имя: "Анна")

Пользователь (ID: 2, Имя: "Иван")

Город (ID: 3, Название: "Москва")

(Рёбра)

Дружба (Пользователь 1 -> Пользователь 2)

Проживание (Пользователь 1 -> Город 3)

Основные элементы графовой модели

1. **Узлы (nodes).** Представляют собой сущности или объекты данных. Например, в социальной сети узлом может быть пользователь, а в системе управления запасами — товар.
2. **Рёбра (edges).** Отображают взаимосвязи между узлами. Они могут быть направленными или ненаправленными и содержать свойства, описывающие характер связи. Например, рёбра могут указывать на дружбу между пользователями или связь между товаром и его категорией.
3. **Свойства (properties).** Узлы и рёбра могут иметь дополнительные атрибуты, которые хранят информацию о них. Например, узел пользователя может иметь свойства, такие как имя и возраст, а ребро дружбы — дату установления связи.
4. **Метки (labels).** Метки позволяют классифицировать узлы и рёбра на основе их типа или назначения. Например, вершины могут быть помечены как «пользователь», «город» или «продукт», а рёбра — как «дружит с», «живёт в» и так далее.

Преимущества

- **Моделирование сложных отношений.** Графовые базы данных идеально подходят для моделирования данных, где отношения между сущностями играют важную роль. Например, они могут использоваться для анализа социальных сетей, рекомендательных систем и семантических сетей.
- **Гибкий и интуитивный запросы.** Графовые базы данных предоставляют интуитивные средства запросов, которые позволяют легко находить и анализировать данные в графе. Языки запросов, такие как Cypher, спроектированы специально для работы с графами.
- **Высокая производительность для определённых задач.** В задачах, где необходимо находить связи между вершинами, графовые базы данных могут обеспечивать высокую производительность и эффективность.

Ограничения

- **Неэффективность для определённых запросов.** Графовые базы данных могут оказаться неэффективными для запросов, которые не связаны с анализом отношений между сущностями.
- **Сложность моделирования данных.** В случае когда структура данных не имеет выраженных отношений, использование графовой модели может быть излишним и усложнить разработку и обслуживание приложения.
- **Сложность масштабирования.** Масштабирование графовых баз данных может оказаться сложной задачей, особенно в случае больших и высоконагруженных графов.

Примеры использования

- **Социальные сети.** Моделирование пользователей и их отношений (друзья, подписчики).
- **Рекомендательные системы.** Предложение товаров или контента на основе анализа связей между пользователями и продуктами.
- **Обнаружение мошенничества.** Анализ транзакций для выявления подозрительных паттернов.

6. Документоориентированная модель

Это подход к организации и хранению данных, в котором информация сохраняется в виде документов. Эти документы могут иметь разнообразную структуру и содержать вложенные данные, что делает модель гибкой и подходящей для работы с полуструктурированными данными. Документные базы данных обычно используют форматы, такие как JSON (JavaScript Object Notation) или BSON (Binary JSON).

ID: завтрак	ID: ланч	ID: обед
<pre>{ "type": "тост", "хлеб": "цельнозерновой", "намазка": ["масло", "джем"] }</pre>	<pre>{ "тип": "салат", "вегетарианский": нет, "ингредиенты": ["шпинат", "помидоры", "огурцы", "морковь", "заправка": ["оливковое масло", "уксус", "мёд", "лимон", "соль", "перец",], "тунец", "грецкие орехи",], "рейтинг": "5 звёзд", "ресторан": "Skylight Diner" }</pre>	<pre>{ "тип": "пицца", "размер": "большой", "топпинги": ["пепперони", "томаты", "сосиски"], "цена": 9.00, "порезанная": да }</pre>

Основные характеристики

1. **Документы.** Основная единица хранения данных — документ. Каждый документ представляет собой набор пар «Ключ — значение» и может содержать вложенные объекты и массивы. Это позволяет хранить сложные структуры данных.
2. **Гибкая схема.** В отличие от реляционных баз данных, где структура данных строго определена, в документоориентированных базах данные могут иметь разные структуры в одной и той же коллекции. Это позволяет легко изменять и обновлять данные без необходимости изменения всей схемы.
3. **Уникальные идентификаторы.** Каждый документ имеет уникальный идентификатор, который обеспечивает быстрый доступ к данным.
4. **Коллекции.** Документы группируются в коллекции, которые можно рассматривать как аналог таблиц в реляционных базах данных. Коллекции могут содержать документы с различной структурой.

Преимущества

- **Гибкость.** Схема данных является гибкой, что позволяет хранить и изменять данные без необходимости изменения всей схемы. Это особенно полезно в ситуациях, когда структура данных меняется с течением времени.
- **Мощные запросы.** Документоориентированные базы данных предоставляют мощные инструменты для запросов данных, включая возможность индексации, фильтрации и агрегации данных.
- **Скорость доступа.** Документы могут быть быстро извлечены по уникальному идентификатору, что обеспечивает высокую производительность.
- **Подходит для больших объёмов данных.** Многие документоориентированные базы данных могут масштабироваться для обработки больших объёмов данных и высоких нагрузок.

Ограничения

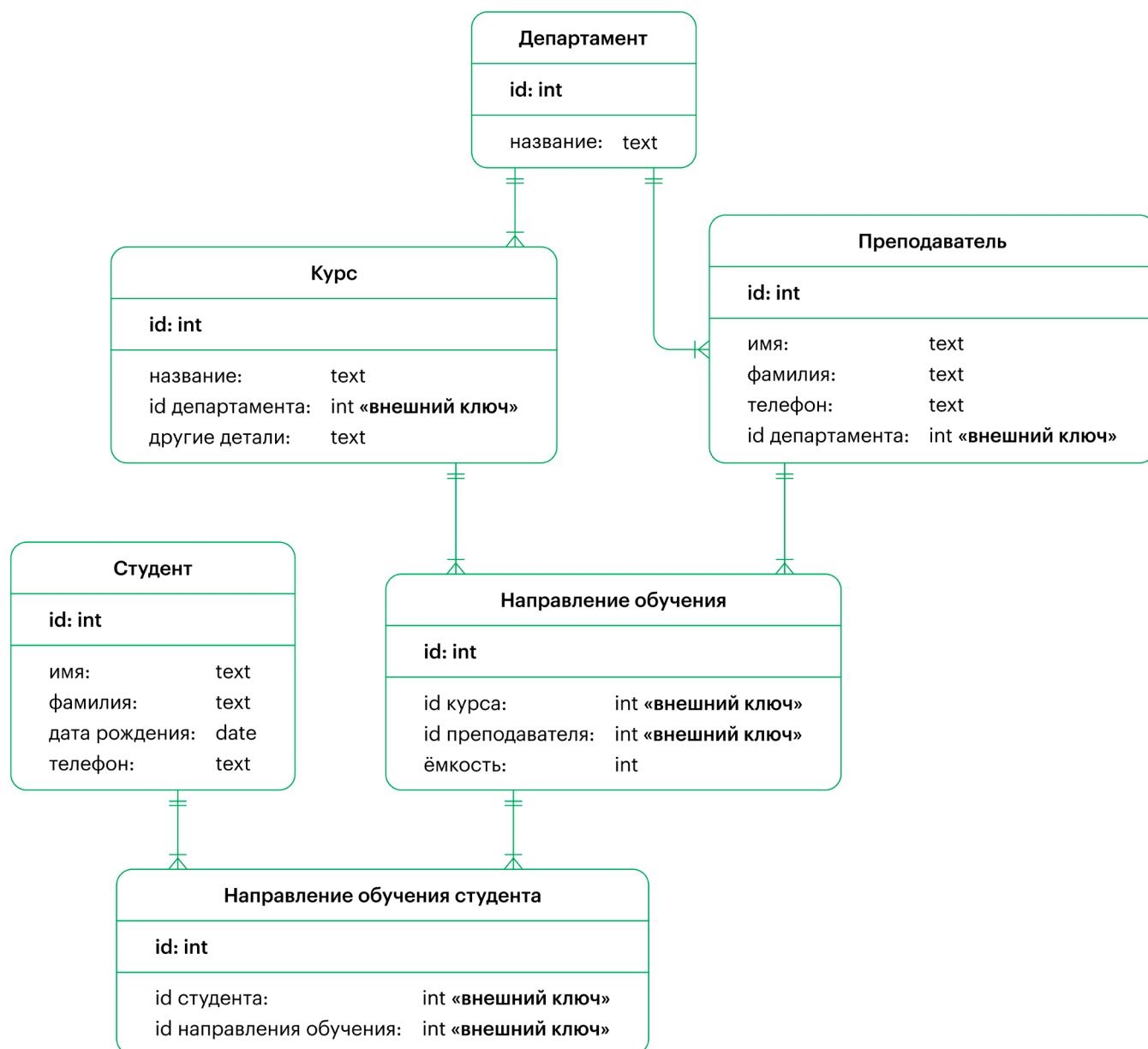
- **Сложность запросов между документами.** В сценариях, где требуется выполнение сложных запросов, связанных с данными из нескольких документов, документоориентированные базы данных могут оказаться менее удобными.
- **Избыточность данных.** Поскольку данные могут дублироваться в разных документах, есть риск избыточности данных и несоответствия данных в разных документах.
- **Не подходит для связанных данных.** В случае когда требуется работать с данными, имеющими сложные взаимосвязи, такие как транзакции и связи между сущностями, модель документов может потребовать дополнительной обработки.

Примеры использования

- **Интернет-магазины.** Позволяет быстро извлекать всю информацию о заказе без необходимости выполнять сложные JOIN-запросы.
- **Блоги, новостные порталы.** Каждая статья хранится как отдельный документ, включая текст, изображения, теги, автора, комментарии и так далее.

7. Реляционная модель

Это подход к организации данных, в котором они представлены в виде таблиц (отношений). Каждая таблица состоит из строк (записей) и столбцов (атрибутов), а данные хранятся в структурированном и взаимосвязанном формате. Эта модель была предложена в 1970 году Эдгаром Коддом в статье «A Relational Model of Data for Large Shared Data Banks» и стала основой для большинства современных баз данных.



Основные элементы реляционной модели

1. **Таблица (Relation, отношение).** Основные структуры хранения данных. Каждая таблица имеет уникальное имя и содержит строки и столбцы.

Пример таблицы «Сотрудники»

ID	Имя	Фамилия	Пол	Дата рождения	ОтделID
1	Сергей	Иванов	М	08.02.1976	1
2	Анатолий	Петров	М	22.06.1998	2
3	Екатерина	Сидорова	Ж	15.10.2003	1

2. **Строки (кортежи).** Каждая строка в таблице представляет собой отдельный объект или запись.
3. **Столбцы (атрибуты).** Каждый столбец содержит данные одного типа и описывает определённый аспект записи, например имя, дату или цену.
4. **Первичный ключ (Primary Key).** Поле или комбинация полей, которые однозначно идентифицируют строку в таблице. В примере выше поле «ID» является первичным ключом.
5. **Внешний ключ (Foreign Key).** Поле или набор полей в одной таблице, который указывает на первичный ключ или поле с ограничением уникальности другой таблицы, создавая связь между ними. Стоит отметить, что внешний ключ может ссылаться на ту же самую таблицу.

Пример: таблица с сотрудниками может содержать поле «ОтделID», указывающее на «ОтделID» из таблицы «Отделы».

Таблица «Отделы»

ОтделID	Название отдела
1	Бухгалтерия
2	Продажи

6. **Язык для работы с данными. Обычно это язык SQL (Structured Query Language).**
Используется для работы с реляционными базами данных и делится на три части:
 - **DDL (Data Definition Language)** — создание и изменение структуры;
 - **DML (Data Manipulation Language)** — управление данными;
 - отдельно выделяют язык запросов — **(DQL (Data Query Language))**;
 - **DCL (Data Control Language)** — управление доступом к данным;
 - **TCL (Transaction Control Language)** — управление транзакциями (и совместной обработкой данных).
 -

Преимущества

- **Структурированность данных.** Данные организованы в чёткой и понятной форме, что упрощает их обработку и анализ.
- **Целостность данных.** Механизмы ограничения целостности помогают предотвратить ввод некорректных данных и поддерживать согласованность между связанными записями.
- **Гибкость запросов.** SQL позволяет легко выполнять сложные запросы для извлечения данных и манипуляции ими.
- **Широкое распространение.** Большинство современных СУБД (MySQL, PostgreSQL, Oracle, SQL Server) основаны на реляционной модели.
- **Масштабируемость.** Реляционные базы данных могут эффективно обрабатывать большие объёмы данных и обеспечивать высокую производительность.

Ограничения

- **Производительность.** При больших объёмах данных сложные запросы могут быть медленными.
- **Гибкость.** Требуется строгая схема данных, что затрудняет работу с неструктурированными данными.

А теперь давай познакомимся с типами данных, которые применяются в SQL.

Типы данных

Числовые типы

Числовые типы включают двух-, четырёх- и восьмибайтные целые, четырёх- и восьмибайтные числа с плавающей точкой, а также десятичные числа с задаваемой точностью.

Имя	Размер	Описание	Диапазон
smallint	2 байта	Целое в небольшом диапазоне	-32768... +32767
integer	4 байта	Типичный выбор для целых чисел	-2147483648... +2147483647
bigint	8 байтов	Целое в большом диапазоне	-9223372036854775808... 9223372036854775807
numeric, decimal	Переменный	Вещественное число с указанной точностью	До 131 072 цифр до десятичной точки и до 16 383 — после
float4, real	4 байта	Вещественное число с переменной точностью	Точность в пределах 6 десятичных цифр
float8, double precision	8 байтов	Вещественное число с переменной точностью	Точность в пределах 15 десятичных цифр
smallserial	2 байта	Небольшое целое с автоувеличением	1... 32767
serial	4 байта	Целое с автоувеличением	1... 2147483647
bigserial	8 байтов	Большое целое с автоувеличением	1... 9223372036854775807

Подробная информация о числовых типах данных.

Целочисленные типы

Типы **smallint**, **integer** и **bigint** хранят целые числа, то есть числа без дробной части, имеющие разные допустимые диапазоны. Попытка сохранить значение, выходящее за рамки диапазона, приведёт к ошибке.

Чаще всего используется тип **integer**, это наиболее сбалансированный выбор ширины диапазона, размера и быстродействия. Тип **smallint** обычно применяется, только когда крайне важно уменьшить размер данных на диске. Тип **bigint** предназначен для тех случаев, когда числа не умещаются в диапазон типа **integer**.

В SQL определены только типы **integer** (или **int**), **smallint** и **bigint**. Имена типов **int2**, **int4** и **int8** выходят за рамки стандарта PostgreSQL, хотя могут работать и в некоторых других СУБД.

Числа с фиксированной точностью

Тип **numeric** позволяет хранить числа с очень большим количеством цифр. Он рекомендуется для хранения денежных сумм и других величин, где важна точность. Вычисления с типом **numeric** дают точные результаты, где это возможно, например, при сложении, вычитании и умножении. Однако операции со значениями **numeric** выполняются гораздо медленнее, чем с целыми числами или типами с плавающей точкой.

NUMERIC (точность, масштаб)

Масштаб значения **numeric** определяет количество десятичных цифр в дробной части, справа от десятичной точки, а **точность** — общее количество значимых цифр в числе, то есть количество цифр по обе стороны от десятичной точки. Например, число 23,5141 имеет точность 6 и масштаб 4. Целочисленные значения можно считать числами с масштабом 0.

Типы **numeric** и **numeric** равнозначны.

Типы с плавающей точкой

Тип данных с плавающей точкой в PostgreSQL используется для хранения чисел с дробной частью (то есть вещественных чисел), которые могут иметь различную степень точности. Этот тип данных применим, если необходимо представлять числа с высокой степенью детализации или для вычислений, где результатом могут быть дробные значения.

Если требуется высокая точность (например, для денежных операций), лучше использовать тип **numeric** или **numeric**, так как они хранят числа с фиксированной запятой.

Типы данных **real** и **double precision** хранят приближённые числовые значения с переменной точностью.

PostgreSQL также поддерживает форматы **float** и **float(p)**, оговорённые в стандарте SQL, для указания неточных числовых типов. Здесь **p** определяет минимально допустимую точность в двоичных цифрах. PostgreSQL воспринимает запись от **float(1)** до **float(25)** как выбор

типа `double precision`. Значения `p` вне допустимого диапазона вызывают ошибку. Если `float` указывается без точности, подразумевается тип `double precision`.

Столбцы с автоматическим увеличением

Типы данных `smallserial`, `serial` и `bigserial` не являются настоящими типами, а представляют собой просто удобное средство для создания столбцов с уникальными числами.

Имена типов `serial` и `serial4` равнозначны: они создают столбцы `integer`. Являются синонимами и имена `bigserial` и `serial8`, но они создают столбцы `bigint`.

Строковые типы

Имя	Описание
<code>character varying(n)</code> , <code>varchar(n)</code>	Строка ограниченной переменной длины
<code>character(n)</code> , <code>char(n)</code> , <code>bpchar(n)</code>	Строка фиксированной длины, дополненная пробелами
<code>text</code>	Строка неограниченной переменной длины

Подробнее о символьных типах данных

SQL определяет два основных символьных типа: `character varying(n)` и `character(n)`, где `n` — положительное число. Оба этих типа могут хранить текстовые строки длиной до `n` символов (не байтов).

Если длина сохраняемой строки оказывается меньше объявленной, значения типа `character` будут дополняться пробелами, а тип `character varying` просто сохранит короткую строку.

Помимо этого, в PostgreSQL есть тип `text`, в котором можно хранить строки произвольной длины. Хотя тип `text` не описан в стандарте SQL, его поддерживают и некоторые другие СУБД SQL. Тип `text` — это встроенный строковый тип данных PostgreSQL, и большинство встроенных функций, работающих со строками, объявляются принимающими или возвращающими `text`, а не `character varying`.

Имя типа `varchar` является псевдонимом для `character varying`, а `char` и `bpchar` (с указанием длины) — для `character`. Псевдонимы `varchar` и `char` определены в стандарте SQL, но `bpchar` — это расширение PostgreSQL.

Если `n` задано, оно должно быть больше нуля и меньше или равно 10485760. При использовании типов `character varying` (или `varchar`) без указания длины принимаются строки любой длины. При использовании типа `bpchar` без указания длины также принимаются строки любой длины, но конечные пробелы семантически не учитываются. Записи `character` (или `char`) без указания длины соответствуют `character(1)`.

Для хранения короткой строки (до 126 байтов) требуется дополнительно 1 байт плюс размер самой строки, включая дополняющие пробелы для типа `character`. Для строк длиннее требуется не 1, а 4 дополнительных байта. СУБД может автоматически сжимать длинные строки, так что физический размер на диске может быть меньше. Очень длинные текстовые строки

переносятся в отдельные структуры, чтобы они не замедляли работу с другими столбцами. В любом случае максимально возможный размер строки составляет около 1 Гб. (Допустимое значение **n** в объявлении типа данных меньше этого числа. Это объясняется тем, что, в зависимости от кодировки, каждый символ может занимать несколько байтов. Если ты хочешь сохранять строки без определённого предела длины, используй типы **text** или **character varying** без указания длины, а не задавай какое-либо большое максимальное значение.

Имя	Размер	Описание	Наименьшее значение	Наибольшее значение	Точность
timestamp [(p)] [without time zone]	8 байтов	Дата и время (без часового пояса)	4713 до н. э.	294 276 н. э.	1 микросекунда
timestamp [(p)] with time zone	8 байтов	Дата и время (с часовым поясом)	4713 до н. э.	294 276 н. э.	1 микросекунда
date	4 байта	Дата (без времени суток)	4713 до н. э.	5 874 897 н. э.	1 день
time [(p)] [without time zone]	8 байтов	Время суток (без даты)	00:00:00	24:00:00	1 микросекунда
time [(p)] with time zone	12 байтов	Время дня (без даты) с часовым поясом	00:00:00 + 1559	24:00:00 – 1559	1 микросекунда
interval [поля] [p) \]	16 байтов	Временной интервал	–178 000 000 лет	178 000 000 лет	1 микросекунда

Подробнее о типах даты/времени

Стандарт SQL требует, чтобы тип **timestamp** подразумевал **timestamp without time zone** (время без часового пояса), и PostgreSQL следует этому. Для краткости **timestamp with time zone** можно записать как **timestamp**tz — это расширение PostgreSQL.

Типы **time**, **timestamp** и **interval** принимают необязательное значение точности **p**, определяющее, сколько знаков после точки должно сохраняться в секундах. По умолчанию точность не ограничивается. Допустимые значения **p** лежат в диапазоне от 0 до 6.

1999-01-08 04:05:06

Или с указанием смещения часового пояса от UTC.

1999-01-08 04:05:06 -8:00

Логический тип

В PostgreSQL есть стандартный SQL-тип `boolean`. Тип `boolean` может иметь следующие состояния: «true», «false» и третье состояние, «unknown», которое представляется SQL-значением NULL.

Имя	Размер	Описание
<code>boolean</code>	1 байт	Состояние: истина или ложь. Значения: true, false

Подробнее о логических типах

Следующие значения соответствуют «true».

```
true
yes
on
1
t
```

А эти значения равны «false».

```
false
no
off
0
f
```

Ключевые слова TRUE и FALSE являются предпочтительными (соответствующими стандарту SQL) для записи логических констант в SQL-запросах.

Полезные ссылки и используемые источники

<https://postgrespro.ru/docs/postgresql/17/datatype>